Source-grounded interview handbook

Indian Contract Intelligence Agent

Interview Preparation and Understanding Guide

Architecture, implementation, trade-offs, failure modes, and interview answers

Prepared from the repository at the current implementation checkpoint
26 September 2026

Source of truth: executable code, migrations, tests, and current deployment files

Contents

1. Executive Overview
2. Problem, Requirements, and Scope
3. Architecture and Components
4. Technology Choices
5. Repository Map
6. End-to-End Workflows
7. Feature Deep Dives
8. Database and Data Model
9. API Deep Dive
10. Caching, Jobs, and State
11. Retrieval, RAG, LLM, and Agents
12. Concurrency and Transactions
13. Failure Handling
14. Security and Trust Boundaries
15. Performance
16. Scalability
17. Why We Built It This Way
18. What I Would Improve
19. Interview Question Bank
20. Model Interview Answers
21. Whiteboard Explanation
22. Explain It Without Code
23. One-Page Revision Sheet

> **Reading rule.** “Implemented” describes current runtime behavior. “Documented checkpoint” describes a recorded verification result, not a fresh run. “Improvement” is a recommendation, not an existing feature.

1. Executive Overview

What the project is

Indian Contract Intelligence Agent (ICIA) is a Python 3.12/FastAPI modular monolith for tenant-scoped contract intelligence. It accepts PDF and DOCX contracts, preserves immutable document versions and provenance, indexes their contents, retrieves relevant evidence, produces evidence-backed answers, extracts structured contract facts, flags deterministic product-risk signals, compares analyzed versions, and can retrieve separately versioned legal-corpus evidence through a bounded agent.

The system is not an autonomous legal-advice system. It reports what a contract or authorized source states, exposes evidence, and labels risk as a product classification. It does not decide legality, validity, enforceability, or legal liability.

> **30-second answer.** “I built a production-oriented contract intelligence backend. A user uploads a PDF or DOCX; the system validates and versions it, extracts structured text and provenance, creates hierarchy-aware chunks and 384-dimensional E5 embeddings, and stores them in PostgreSQL with pgvector. Search combines dense vectors with request-scoped BM25 and optional local reranking. Chat is fail-closed: the model receives bounded, tenant-authorized evidence and citations are verified by the application. Separate deterministic services extract clauses, obligations, and risk signals, compare contract versions, and support a bounded contract-plus-legal agent.”

> **2-minute answer.** “The interesting engineering problem is not just calling an LLM. It is preserving trust from upload to answer. Upload validation checks type, signature, size, filename, checksum, and duplicate versions. Processing preserves page, section, clause, block, and extraction-method provenance. Every retrieval request is tenant-scoped before evidence is exposed. Dense retrieval uses pgvector; sparse retrieval preserves legal identifiers and numbers; hybrid search uses reciprocal-rank fusion; an optional cross-encoder reranks only retrieved candidates. Generation gets application-assigned E1-style evidence IDs, a context budget, structured JSON output, and fail-closed citation validation. Contract analysis is mostly deterministic and persists a replaceable derived snapshot keyed by version and configuration hash. Auth uses signed access tokens, scrypt password hashes, and tenant memberships. The main trade-off is a simple modular monolith with synchronous CPU-heavy processing: it is understandable and easy to operate, but needs queues, object storage, distributed metrics, and scaling work for larger traffic.”

> **5-minute answer structure.** Start with the user problem, draw the upload-to-index path, then draw search-to-answer. Explain tenant scope and evidence as the safety boundaries. Add the immutable document/version model, chunks with provenance and vectors, derived analysis/comparison snapshots, and separate legal corpus tables. Close with the current deployment shape, synchronous processing bottleneck, and the next scaling step: durable jobs plus isolated model workers.

Why it is technically interesting

- It combines transactional data, immutable versioning, vector search, lexical search, reranking, structured extraction, and bounded agent orchestration.
- It treats provenance, tenant isolation, and citation validation as application invariants rather than prompt instructions.
- It deliberately keeps contract analysis deterministic where possible and isolates optional LLM semantics from canonical state.
- It contains realistic operational constraints: local model memory, OCR subprocess bounds, synchronous work, configuration hashing, and incomplete production-scale measurement.

2. Problem, Requirements, and Scope

The problem

Contracts are long, structurally irregular, versioned, and full of high-value details such as notice windows, renewal mechanics, liability caps, obligations, payment terms, and governing-law clauses. Keyword search misses paraphrases; a raw LLM call can hallucinate; a flat text dump loses page and clause provenance; and a multi-tenant product cannot trust caller-supplied tenant identifiers.

Requirements derived from the implementation

Category	Must have in current system	Current boundary
Functional	Upload PDF/DOCX; immutable versions; parse/OCR; chunk/embed; dense/sparse/hybrid search; evidence-backed chat; analysis; comparison; legal retrieval; bounded agent; auth/RBAC; UI.	Conversation persistence, durable agent runs, and standalone legal JSON search are absent.
Reliability	Validation, checksums, transactional indexing, explicit FAILED state, structured errors, fail-closed citations, deterministic risk rules.	Upload/processing work is synchronous; no queue-based recovery.
Security	Signed tenant context, membership and roles, scrypt passwords, bounded subprocesses, path-safe storage, scope-filtered evidence, prompt-injection boundaries.	No MFA, SSO, refresh-token rotation, rate limiter, or general policy engine.
Performance	pgvector HNSW, embedding reuse, candidate bounds, context limits, CPU model cache.	BM25 scans authorized chunks per request; reranking and OCR consume API CPU.
Consistency	Database constraints, immutable source versions, configuration-hashed derived snapshots, tenant filters.	Local object storage and database updates are not a distributed transaction.
Scalability	Containerized topology and separated domain services inside one process.	No worker queue, autoscaling plan, replicas, sharding, or durable metrics backend.

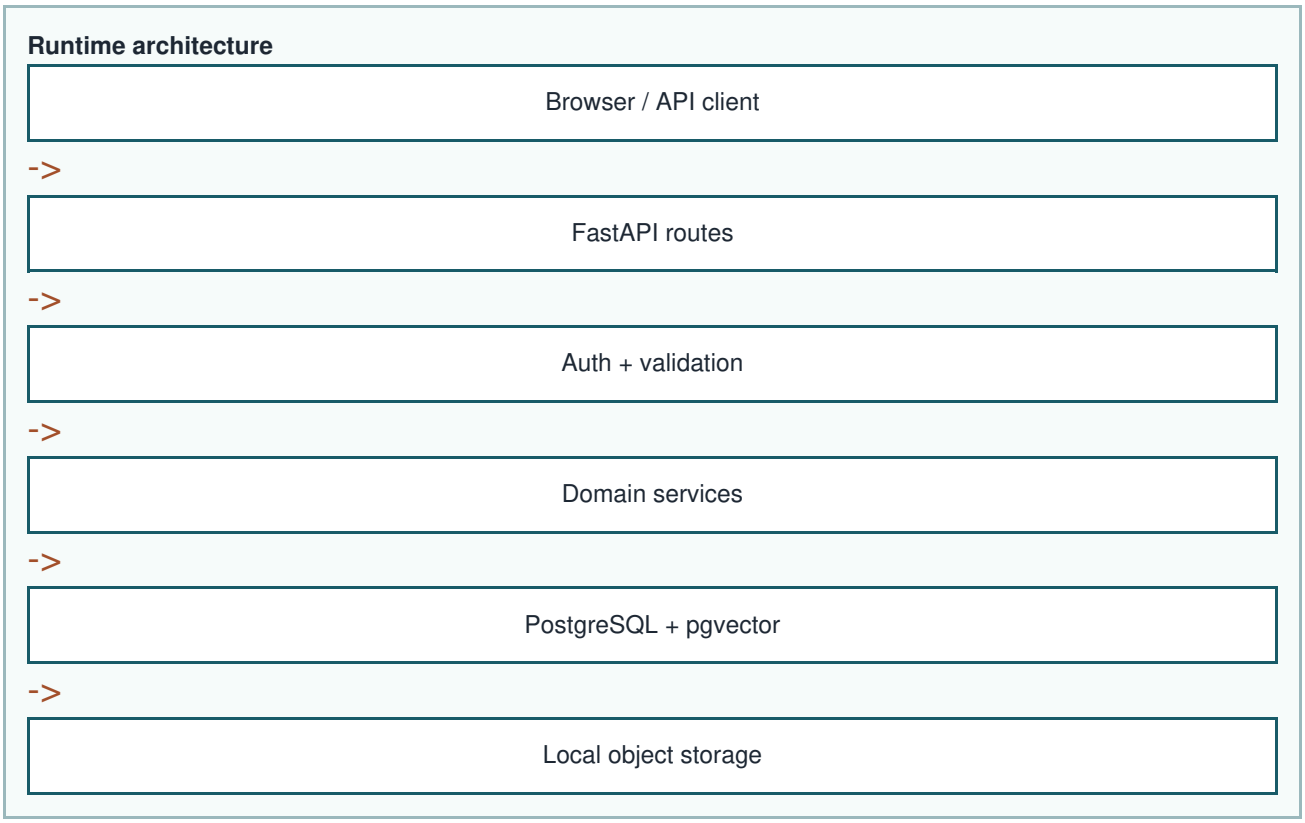
Naive approach and why it fails

A naive design stores uploaded text in one table and sends the entire contract to an LLM. It fails on context limits, latency, OCR and page provenance, access isolation, reproducibility, citation trust, duplicate uploads, and version comparison. ICIA answers these by separating immutable sources from derived artifacts, retrieving bounded evidence, validating structured output, and making scope explicit at every boundary.

Scope language for an interview

> **Do not overclaim.** The repository provisions Redis but does not use Redis reads/writes for application caching. It has a legal corpus and legal retrieval, but legal data is separate and not automatically authoritative. The recorded 54-test checkpoint and Compose results are historical documentation; they are not a fresh verification performed for this PDF.

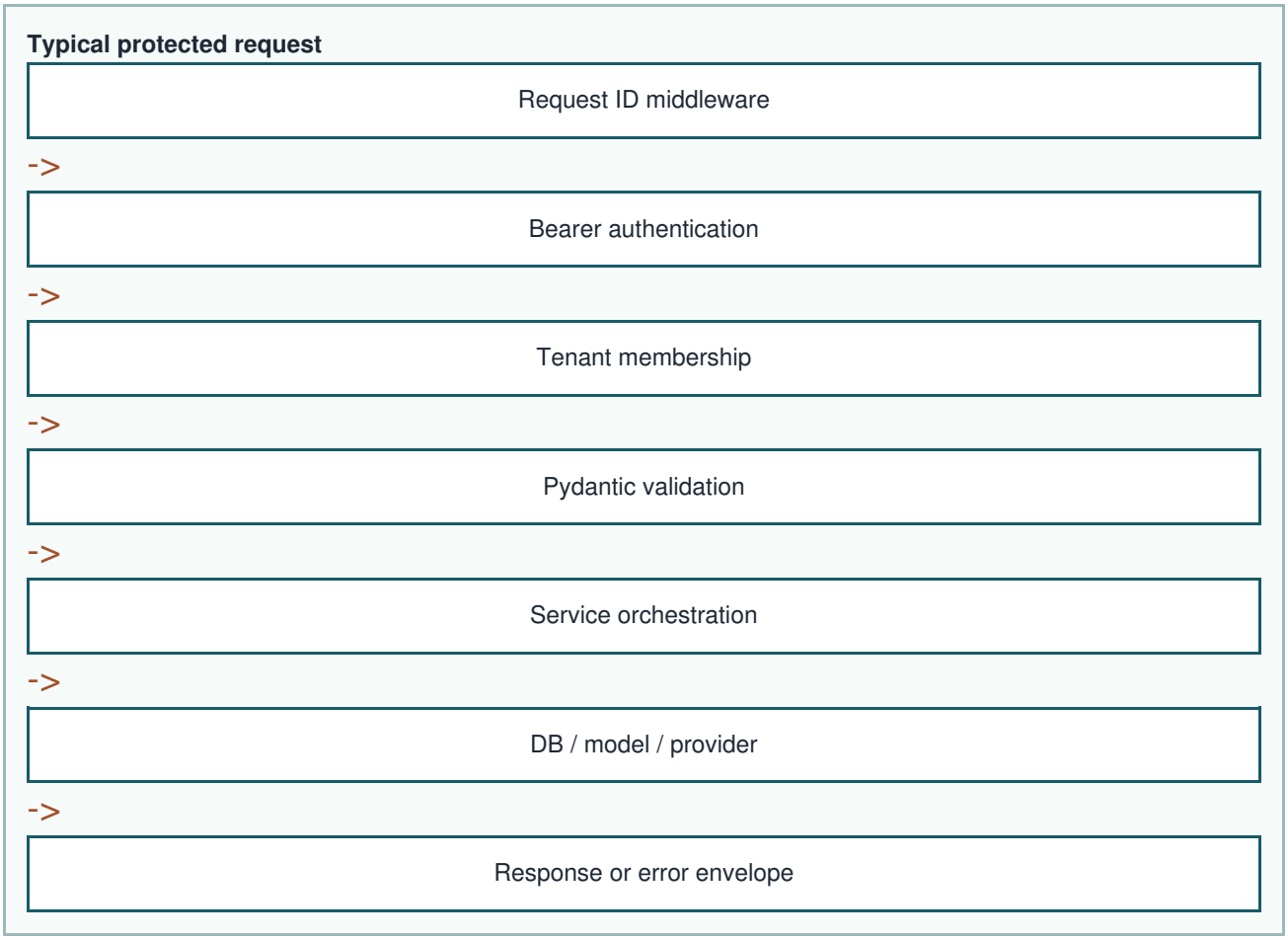
3. Architecture and Components



Optional paths attach to the domain services: FastEmbed embeddings and the local ONNX reranker run in-process; Groq is the external LLM provider; Tesseract and optional LibreOffice are bounded subprocesses; Redis is provisioned but outside current application semantics; Nginx terminates TLS in production.

Component	Receives / produces	Why it exists
FastAPI + Jinja UI	HTTP and multipart requests / JSON or server-rendered pages	One deployable application boundary with typed request validation.
Auth context	Bearer token, optional tenant header / user, tenant, membership, role	Makes signed tenant scope authoritative and centralizes permission checks.
Ingestion	PDF/DOCX bytes / original object, DocumentVersion, ProcessedDocument	Separates validation, storage, parsing, and lifecycle state.
Indexing	ProcessedDocument / hierarchical Chunk rows and vectors	Preserves structure and makes retrieval efficient.
Retrieval	Query plus tenant/document filters / ranked evidence with diagnostics	Combines semantic and exact-match strengths.
Generation	Authorized evidence / structured answer and verified citations	Uses the LLM as an untrusted component.
Contract services	READY version and chunks / analysis, risks, comparisons	Keeps deterministic product logic out of prompts.
Legal and agent services	Corpus-scoped query or typed task / legal evidence or bounded answer	Prevents contract and legal corpora from being mixed implicitly.
PostgreSQL	Relational rows and vectors / durable state	One transactional system for metadata, chunks, auth, and vectors.
Local storage	Generated safe storage keys / original and processed artifacts	Keeps large files out of relational rows in the current deployment.

Request lifecycle



Unexpected exceptions are logged with a category and become a generic 500 containing a request ID. Expected [illegible] instances retain a status and stable error code.

4. Technology Choices

Technology	Where used	Why selected	Alternative and trade-off
Python 3.12 + uv	Application and dependency management	Strong data/AI ecosystem, typing, and reproducible lockfile workflow.	Node or Java could serve APIs; Python makes local inference and document tooling direct, but CPU-bound work needs isolation.
FastAPI + Pydantic v2	Routes, schemas, validation	Typed async HTTP boundary and explicit models with forbidden extra fields in key domains.	Django would provide more batteries; FastAPI keeps the modular service boundary lighter.
PostgreSQL 16	Metadata, auth, chunks, snapshots, legal data	Transactions, constraints, joins, JSON, and mature operational model.	MongoDB would simplify documents but weaken relational integrity; a second vector DB would add operational and consistency cost.
pgvector	Dense embeddings and HNSW cosine index	Keeps vector retrieval beside tenant/version filters and transactional metadata.	Dedicated vector DB may scale independently later, but adds synchronization and another failure surface.
SQLAlchemy 2 + Alembic	ORM and migrations	Typed relational access and versioned schema evolution.	Raw SQL can be more direct; ORM conventions reduce repeated mapping but require query discipline.
FastEmbed E5 small	384-dim CPU embeddings	Local, cacheable, multilingual retrieval with query/passage prefixes.	Hosted embeddings can improve quality but add cost, latency, and data egress.
BM25 + RRF	Sparse and hybrid search	Exact identifiers and semantic paraphrases complement each other without a second search service.	Persistent inverted index would scale better than per-request materialization.
FastEmbed ONNX cross-encoder	Optional reranking	Improves ordering after bounded candidate retrieval while staying local.	Remote reranking can use stronger models but increases latency and cost; measured reports show roughly 2.6-4.5s means on small experiments.
Groq provider	Optional semantic classification and chat	Existing provider abstraction, structured JSON, bounded timeout/retries.	A local LLM reduces egress but raises memory/latency; any provider remains untrusted.
Docker Compose + Nginx	Development and one-VM production topology	Repeatable PostgreSQL/Redis/API deployment and TLS proxy.	Kubernetes would improve orchestration at scale but is explicitly deferred.
Redis 7	Provisioned service and client lifecycle	Prepared for ephemeral infrastructure.	It is not currently a cache or readiness dependency, so claiming cache behavior would be inaccurate.

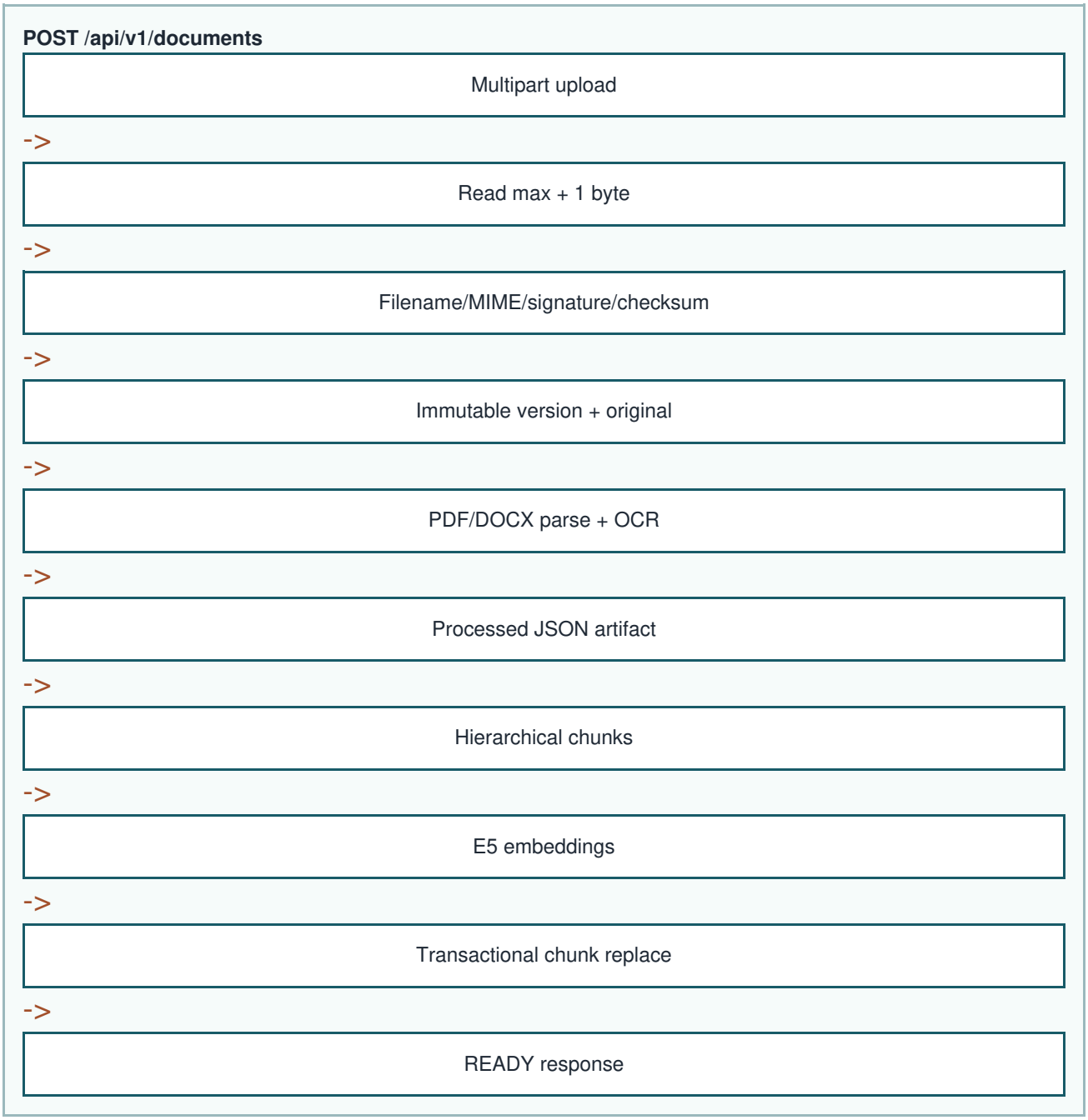
5. Repository Map

Path	Architectural role
app/main.py	Application lifespan, dependency construction, exception handlers, request middleware.
app/api/v1/endpoints/	HTTP boundary: auth, documents, search, chat, agent, contracts, admin, health, UI.
app/security/	Token verification, password hashing, current context, permission dependencies.
app/services/ingestion/	Upload validation, local storage, PDF/DOCX parsing, OCR, processing lifecycle.
app/services/indexing/	Structure-aware chunking, deterministic identities, embeddings, transactional indexing.
app/services/retrieval/ and reranking/	Query normalization, dense search, BM25, RRF, optional cross-encoder.
app/services/generation/	Context budget, provider calls, structured output, citation verification.
app/services/contracts/	Schema, deterministic extraction, evidence checks, risks, comparison.
app/services/legal/ and agents/	Separate corpus retrieval and bounded typed tool orchestration.
app/models/ and migrations/	SQLAlchemy entities and Alembic schema history 0001 through 0008.
templates/	Current server-rendered product UI; no Next.js frontend is implemented.
tests/	Unit, integration, and evaluation coverage for behavior, persistence, retrieval, agents, UI, and metrics.
docker-compose*.yml, Dockerfile, docker/ nginx/	Local and one-VM production deployment, OCR/runtime dependencies, TLS proxy.
docs/architecture, docs/decisions, docs/ evaluation	Design rationale, operating guidance, and benchmark artifacts; some older phase text is stale.

> **Documentation conflict to mention.** README and older architecture files still describe Next.js, future authentication, or earlier phase checkpoints. Current code has Jinja UI, auth/RBAC, comparison, legal retrieval, and agents. No .git directory is present in this checkout, so commit history could not be verified.

6. End-to-End Workflows

Upload, processing, and indexing



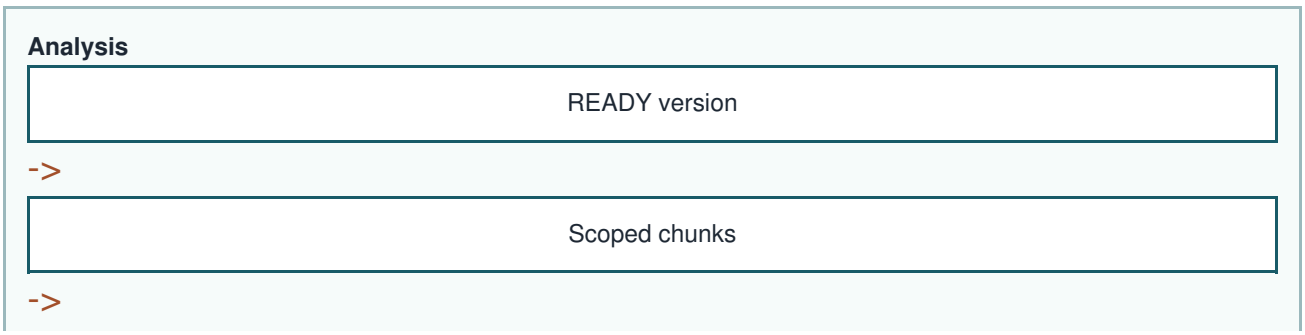
1. Authentication supplies the authoritative tenant; a body tenant ID is ignored.
2. The bounded read detects oversize uploads before expensive processing. Extension, MIME, magic bytes, filename safety, emptiness, and checksum are validated.
3. Storage keys are generated from IDs, not user paths. Same-title identical content is rejected by checksum uniqueness; changed content receives a new version.
4. PDF native blocks are preferred; low-text/image pages can use Tesseract within a timeout. DOCX uses python-docx; physical pages are not invented when conversion is unavailable.
5. Indexing preserves sections, clauses, pages, printed labels, extraction method, and source-block provenance. Embeddings are checked for finite values and dimension 384.
6. Failure marks the version FAILED; the indexing transaction rolls back partial chunk changes.

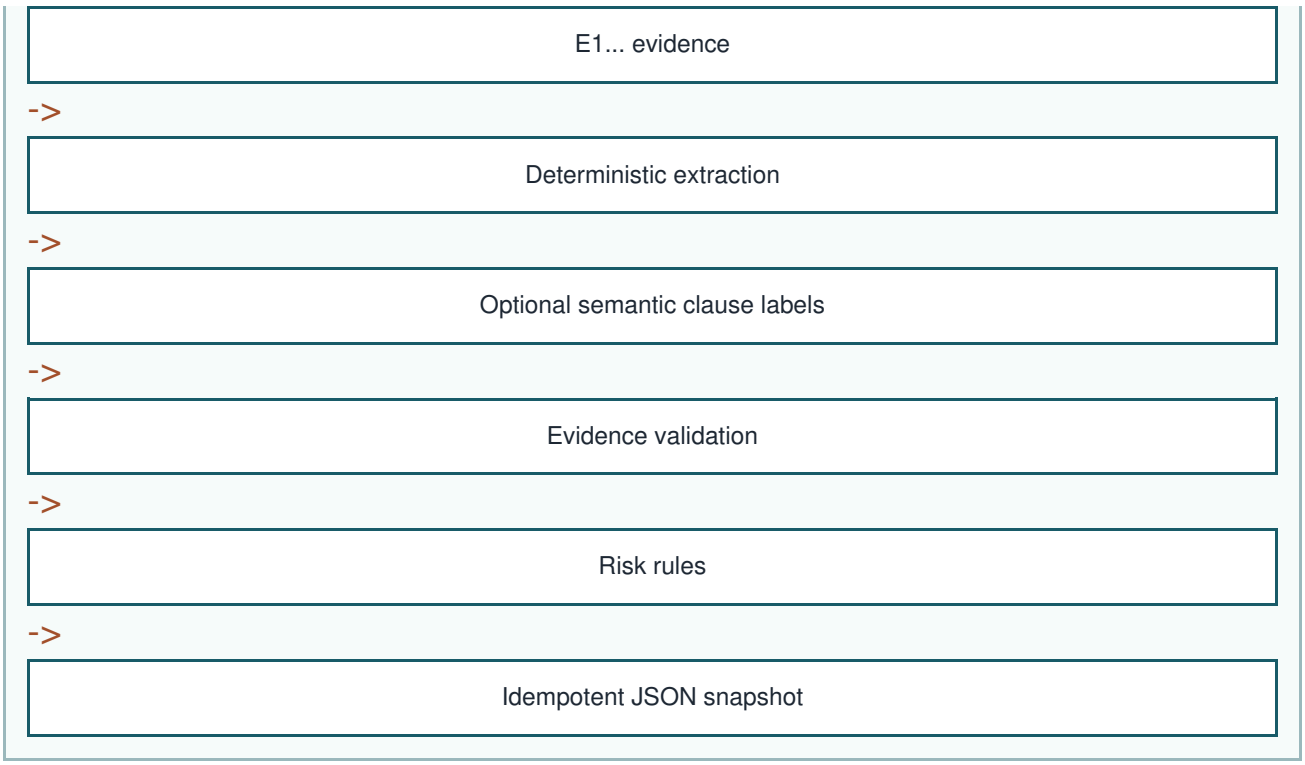
Search and chat



Search can return diagnostics even when generation is not safe. Chat requires explicit reranking, a usable reranker score, and authorized evidence; otherwise it returns an insufficient-evidence answer without calling the LLM. The model never supplies citation metadata: the server copies it from the authorized registry.

Contract analysis and comparison





Comparison requires two authorized READY, already analyzed versions. It is deterministic and does not call Groq. Both analysis and comparison are derived snapshots, while source versions remain immutable.

Agent query

The agent accepts typed plans and exposes only `contract_search`, `contract_analysis`, `contract_comparison`, and `legal_search`. It has bounded steps, dependency checks, duplicate-call and cycle detection, timeouts, and separate contract/legal evidence namespaces. It has no SQL, shell, filesystem, arbitrary HTTP, Redis, or code-execution tool.

7. Feature Deep Dives

Document intelligence

What/why: turn messy source files into traceable structured artifacts. **Internal decision:** structural boundaries take precedence over token boundaries; headings, clause numbers, and provenance survive chunking. **Edge cases:** corrupt files, MIME/signature mismatch, empty files, OCR timeout, low-text pages, missing printed labels, duplicate checksum, and partial indexing. **Interview point:** “READY means processing and indexing completed, not merely that bytes were uploaded.”

Retrieval

What/why: dense embeddings capture paraphrase; BM25 preserves exact legal numbers and identifiers; RRF avoids forcing incomparable score scales into one formula. Candidate size is bounded, tenant predicates are applied in SQL first, and tie-breaking is deterministic. **Trade-off:** BM25 materializes the authorized chunk set per request, which is simple and auditable but a likely scale bottleneck.

Contract extraction and risk

What/why: stable facts and risk signals should not depend on an LLM response. Deterministic extraction is authoritative for identifiers and provenance; semantic classification is optional and narrowly scoped. Evidence IDs and confidence values are validated. Rules include automatic renewal, short cancellation windows (<=30 days), long termination notice (>=90 days), long confidentiality survival (>=5 years), uncapped liability, indemnity, payment, IP, restrictive covenant, missing clause, and deadline signals. These are product-risk findings, not legal conclusions.

Legal retrieval

Legal corpus versions, documents, versions, and chunks use separate tables and `L1`-style evidence. Corpus-version scope is mandatory. The separation prevents accidental mixing of contract text and legal source text and makes source release identity explicit.

Server-rendered UI

The current product surface is Jinja-rendered: dashboard, contract workspace, upload, search, Ask ICIA, comparison, legal upload, administration, login, signup, and logout. The README's “Next.js later phase” is not current behavior.

8. Database and Data Model

Core entity relationships

Tenant - User membership

->

Tenant - Documents

->

Document - Versions

->

Version - Chunks

->

Version - Analysis snapshot

->

Version pair - Comparison

->

Corpus version - Legal documents - Legal chunks

Table	Important design	Interview explanation
tenants / users / tenant_memberships	Unique tenant identifier, unique email, unique tenant-user membership, ADMIN/REVIEWER/VIEWER role.	Many-to-many users/tenants is represented explicitly so membership and role are durable and queryable.
documents / document_versions	Version number and checksum unique per document; storage key unique; lifecycle check constraint.	The source is immutable and changed content becomes a new version, enabling reproducibility and comparison.
chunks	Tenant/version identity, hierarchy, text, provenance JSON, 384-vector, content/config hashes; composite FK prevents mismatched document/version.	Retrieval needs both searchable text/vector and trustworthy location metadata.
contract_analyses	Tenant/version, analysis version, configuration hash, JSON payload; unique configuration key.	Derived state can be replaced or regenerated without mutating source documents.
contract_comparisons	Source/target versions, status, configuration hash, JSON payload; unique tenant/pair/configuration.	Deterministic comparison is idempotent and tied to exact versions.
legal_*	Corpus release, legal document/version, chunk and vector tables separate from contracts.	Corpus provenance and scope are first-class, not a mixed search index.

Indexes and constraints

PostgreSQL indexes cover tenant IDs, document/version lookups, version-section-clause filters, and HNSW cosine vectors. Uniqueness prevents duplicate versions, chunk identities, analysis configurations, comparisons, corpus

releases, and memberships. Foreign keys cascade source deletion to derived rows; parent chunks use SET NULL. JSON stores provenance/metadata and derived payloads where fields evolve as a validated Pydantic contract rather than requiring a column for every extracted fact.

Transactions and concurrency

Indexing deletes/rebuilds a version's chunks transactionally; a failure rolls back and marks the version FAILED. Idempotency is database-backed through unique constraints and configuration hashes. A concurrent duplicate upload or analysis can still race before commit; the unique constraint is the final arbiter and the service must map that conflict. There is no distributed transaction joining local files, PostgreSQL, Groq, and model caches, so recovery is operationally important.

Whiteboard questions

At 10x traffic, inspect BM25 scans, vector index memory, connection pool limits, synchronous ingestion, and JSON snapshot size. At 100x, move processing and embedding to durable jobs, use object storage, separate model workers, aggregate metrics, and consider a dedicated sparse/vector system only after measurements justify the added consistency cost.

9. API Deep Dive

Method	Endpoint	Purpose	Important errors / controls
GET	/api/v1/health	Process liveness.	No database or Redis dependency; returns request ID.
GET	/api/v1/ready	Dependency readiness.	Current implementation checks PostgreSQL; 503 when unavailable.
POST	/api/v1/auth/token	Issue bearer token from OAuth2 form.	401 invalid credentials; 403 no membership.
POST	/api/v1/auth/bootstrap	Development/test tenant and first user.	Requires matching bootstrap header; unavailable outside dev/test.
POST	/api/v1/documents	Validate, process, index upload.	422 invalid input; 409 duplicate; failed processing; synchronous latency.
POST	/api/v1/search	Return retrieval results and diagnostics.	422 schema; 403 missing scope; 503 reranker failure.
POST	/api/v1/chat	Generate evidence-backed answer.	503 not configured; 504 timeout; 502 malformed output/citations; 429 rate limit.
POST	/api/v1/contracts/{version}/analyze	Create or replace analysis snapshot.	404 scope; 422 non-READY/artifact; evidence fails closed.
GET	/api/v1/contracts/{version}/analysis|clauses|obligations|risks	Read derived intelligence.	Tenant and version authorization.
POST/GET	/api/v1/contracts/compare[/{id}]	Create/read deterministic comparison.	Requires authorized analyzed versions; idempotent configuration.
POST	/api/v1/agent/query	Run bounded typed multi-tool workflow.	Plan/tool validation, timeouts, dependency/cycle/duplicate checks.
POST/PATCH	/api/v1/admin/...	Manage users, memberships, roles.	Admin permission; final admin cannot be removed/demoted.

All routes live under `/api/v1`. Expected errors have `error.code`, message, and request ID. Payload tenant IDs are not trusted; the signed token's tenant is authoritative.

10. Caching, Jobs, and State

What actually exists

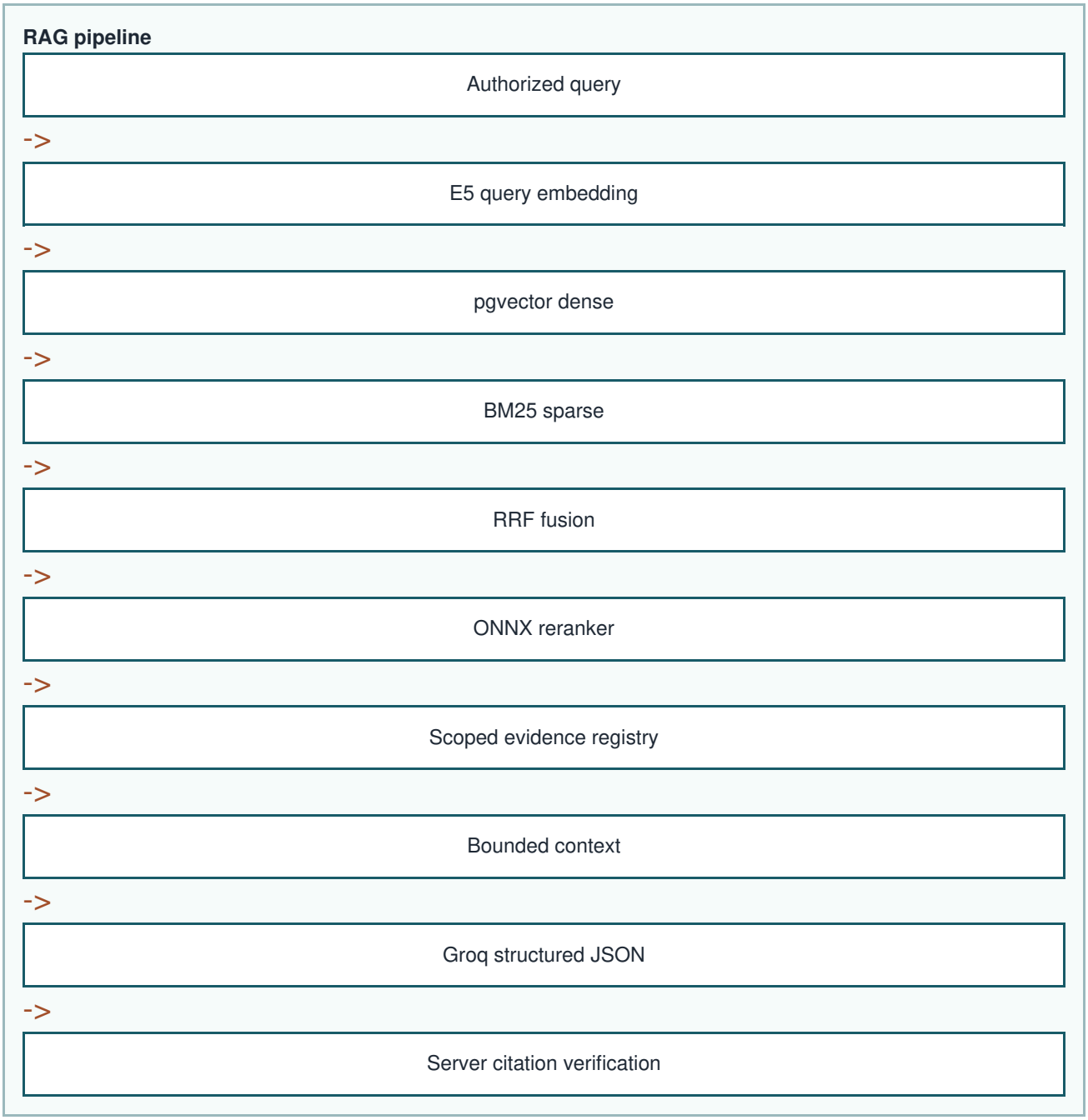
- FastEmbed model files are cached locally in a mounted model-cache volume.
- Embedding reuse is database-backed by content hash and embedding configuration hash.
- SQLAlchemy uses connection pooling and `pool_pre_ping`.
- Redis is created and managed in the resource layer and is provisioned by Compose, but no application cache key is read or written on the request path.
- No Celery/RQ/Kafka worker, job table, conversation table, durable agent-run table, or distributed cache is implemented.

> **Interview correction.** Do not say “Redis caches retrieval.” The accurate answer is: “Redis is provisioned for future ephemeral infrastructure, but current application semantics do not depend on it; readiness intentionally checks PostgreSQL only.”

Why this matters

The absence of a queue means upload, OCR, parsing, chunking, embedding, and indexing occupy the HTTP request. This is understandable for an early modular monolith and simplifies consistency, but it increases timeout risk and makes horizontal scaling harder when model memory and CPU are involved.

11. Retrieval, RAG, LLM, and Agents



Exact mechanics

Documents use the E5 `passage:` prefix and queries use `query:`. Dense scoring is `1 - cosine distance`. BM25 uses `k1=1.2` and `b=0.75`, with normalization that preserves legal identifiers, numbers, hyphenated terms, and Devanagari. Hybrid scoring uses:

`RRF(d) = sum(weight / (rrf_k + rank))`, with default `rrf_k=60` and equal weights. Ties use chunk UUIDs for deterministic output.

Why retrieval is a safety boundary

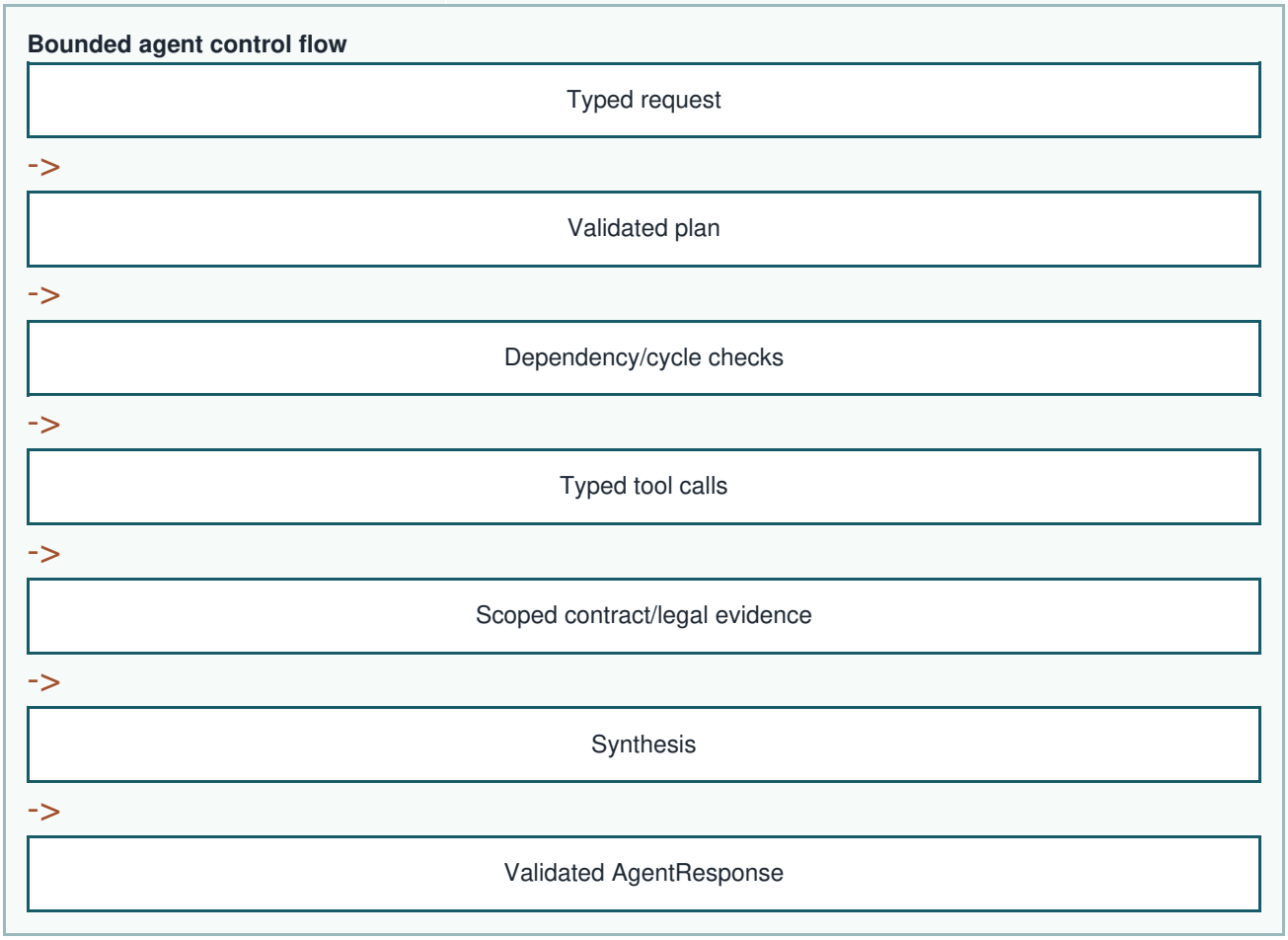
SQL predicates apply tenant/document/version scope before evidence is exposed. The context builder assigns deterministic E1, E2 IDs to the full authorized registry, then selects by item and estimated-token limits. Items excluded

from selected context are not citation-authorized. Unknown or duplicate IDs fail; model-supplied metadata is ignored.

LLM behavior

Groq is called with temperature zero, JSON response format, 30-second default timeout, and up to two retries. Missing configuration, timeout, connection/5xx, rate limit, auth, malformed output, and invalid citation cases map to distinct application errors. No sufficient authorized reranked evidence means no LLM call.

Agent architecture



The agent is orchestration, not unrestricted execution. It has no arbitrary code or network tools. The key trade-off is reduced flexibility in exchange for a smaller attack surface and predictable resource bounds. There is no durable conversational memory.

Quality, cost, and risk

Latency includes database retrieval, local embedding/reranking, network LLM time, and retries. CPU reranking reports are experimental and based on small datasets; they should not be presented as production SLA numbers. Hallucination risk is reduced by evidence gates, structured output, and citation validation, but retrieval incompleteness and corpus quality remain risks.

12. Concurrency and Transactions

Situation	Current mechanism	Residual risk / answer
Duplicate upload	Checksum uniqueness per document plus service duplicate detection.	Concurrent requests may race; DB uniqueness protects final state and should be translated to 409.
Repeated analysis	Unique version/analysis/configuration key and replacement repository behavior.	Derived snapshot is replaceable; source version stays immutable.
Partial indexing	Transactional chunk replacement and FAILED lifecycle.	Original/processed files and DB state require recovery checks because storage is not transactional with DB.
Concurrent reads/writes	PostgreSQL isolation and scoped queries; immutable versions reduce mutation conflicts.	Long synchronous processing still consumes connections and request slots.
Provider retry	Bounded provider retries and typed error classes.	Retries can duplicate external work; generation is not persisted as a durable job.
Multi-process scaling	Not yet established; local model caches and process-local metrics.	Multiple workers duplicate model memory and fragment metrics; production recommends one worker today.

The central consistency strategy is “immutable source plus replaceable derived state.” That is easier to reason about than mutating extracted facts in place and supports regeneration when configuration changes.

13. Failure Handling

Failure	Detection	Handling	User impact / recovery
Invalid upload	Pydantic, filename/MIME/ signature/size checks	Reject before processing; structured status	Client fixes input; no READY version.
Duplicate content	Checksum lookup/unique constraint	DuplicateDocumentError / 409	Existing version remains authoritative.
OCR or parser failure	Subprocess timeout/ exception or invalid artifact	Mark FAILED; return processing error	Retry with corrected file/ configuration; no fabricated provenance.
Database unavailable	SQLAlchemy exception/ readiness	503 readiness or generic categorized 500	Restart/dependency recovery; request ID supports diagnosis.
Redis unavailable	Redis client/health path if touched	Current API semantics do not depend on Redis	Readiness is intentionally PostgreSQL-only.
Reranker unavailable	Explicit RerankingError	503; do not claim successful reranking	Use non-generation search or restore model dependency.
LLM timeout/rate limit/ malformed JSON	Provider typed error mapping	504/429/502 response; no fabricated answer	Retry only within bounded policy or show safe failure.
Unknown/duplicate citation	Citation verifier	Fail closed with 502	Do not show unsupported answer as grounded.
Wrong tenant	Signed tenant and membership checks	401/403 or boundary 404	Do not reveal cross-tenant existence.
Agent cycle/duplicate/ dependency error	Planner/executor validation	Reject or controlled agent failure	No arbitrary tool execution.

Every expected error includes a request ID. Normal logs exclude contract/provider content; errors are categorized as database, Redis, ingestion, auth, citation, provider, or internal without exposing sensitive payloads.

14. Security and Trust Boundaries

Implemented controls

- HS256 signed access tokens include subject, tenant, issue/expiry, issuer, audience, and access-token type.
- Passwords use salted `hashlib.scrypt`; production requires a JWT secret of at least 32 characters and disallows bootstrap configuration.
- Authenticated context verifies active user, tenant, membership, and role. ADMIN, REVIEWER, and VIEWER permissions are explicit.
- Caller tenant fields are overwritten by authenticated scope; optional `X-Tenant-ID` must match the signed claim.
- Generated storage keys and path resolution resist traversal; uploads and subprocesses are bounded.
- SQLAlchemy parameters and Pydantic validation reduce injection and malformed-input risks.
- Retrieved contract text is data, not instructions; the agent exposes typed tools only; evidence/citation checks are application-owned.

Weaknesses to state clearly

No MFA, SSO, refresh-token rotation, email verification, general policy engine, application rate limiter, centralized audit log, or dependency security program is established in the current implementation. Local storage is not object-store replicated. Prompt-injection defense is defense-in-depth, not a claim that malicious text is solved.

15. Performance

Operation	Cost shape / bottleneck	Current mitigation
Upload and parse	O(file size) plus OCR subprocess work; synchronous request.	Size bounds, subprocess timeouts, native extraction before OCR.
Chunking	O(processed text) with structural parsing.	Token targets 600, soft 1000, hard 1500; bounded artifacts.
Embedding	O(number of chunks * model inference); CPU and memory.	Batch size 32, local cache, reuse by content/config hash.
Dense retrieval	Approximate vector search plus filtered relational predicates.	HNSW cosine index and bounded candidate count.
BM25	O(authorized chunks) per request in current implementation.	Candidate/top-k bounds; likely first scale bottleneck.
Reranking	O(candidate count * cross-encoder inference).	Candidate cap 40, batch 8, max length 512, optional flag.
LLM generation	Network latency plus token cost and retries.	Context 6000 estimated tokens, output 1200, timeout/retry bounds.
Analysis/ comparison	O(chunks/facts) deterministic; comparison aligns extracted structures.	Version-scoped inputs and persisted snapshots.

There are no trustworthy project-wide latency or SLA numbers in the current implementation. Dated reranker experiments are small and should be used as directional evidence only.

16. Scalability

Scale	What likely breaks first	Reasonable next step
100 users	Synchronous upload/CPU contention and local model warm-up.	Keep one worker, prewarm models, measure request queues and DB pool use.
10,000 users	BM25 scans, upload timeouts, model CPU, local storage, process-local metrics.	Durable job queue and worker pool; object storage; persistent sparse index; centralized metrics; rate limiting.
1 million users	Single PostgreSQL/vector instance, tenant filtering, snapshot volume, provider cost, operational blast radius.	Read replicas/partitioning after measurements, independently scalable retrieval, sharded or dedicated vector/search systems if justified, autoscaling workers, service extraction only where boundaries earn it.

Current vs proposed: current production topology is one Linux VM, one FastAPI worker, PostgreSQL/pgvector, Redis, and Nginx. Horizontal API replicas are not a free win because local model memory and process-local metrics are not coordinated. The first architectural change should be durable asynchronous processing, not prematurely splitting every service.

17. Why We Built It This Way

Decision	Alternative	Reasoning and trade-off
Modular monolith	Microservices	Shared domain boundaries and one deployment reduce network and operational complexity while the product is still evolving. Service extraction becomes attractive when independent scaling or ownership is measured.
PostgreSQL + pgvector	MongoDB plus vector DB	Relational integrity, tenant filters, metadata, and vectors stay together. A separate vector system may win at scale but creates sync and consistency work.
Evidence-backed generation	Prompt-only citations	Application-owned evidence and metadata make citations verifiable. It constrains model flexibility and requires retrieval/context plumbing.
Immutable versions + derived snapshots	Mutate one contract record	Reproducibility, comparison, and regeneration improve. Storage and cleanup are more involved.
Deterministic extraction first	LLM writes canonical facts	Deterministic provenance and rules are auditable. Semantic coverage is weaker until optional classification is added.
Separate legal corpus	Mix legal and contract chunks	Prevents scope confusion and preserves corpus release provenance. It requires duplicate schema/retrieval concepts.
Explicit tenant context	Trust request tenant ID	Signed membership scope prevents cross-tenant access. It requires auth dependencies on every protected path.
Local CPU models	Hosted embeddings/ reranker	Avoids data egress and recurring inference API cost. CPU latency and memory become operational constraints.
Synchronous first implementation	Queue and workers immediately	Simpler consistency and local development. It is the clearest scale limitation today.

18. What I Would Improve

Current approach	Problem	Proposed improvement	Trade-off
Synchronous upload/ process/index	Long requests and API CPU contention	Durable job table plus worker queue; return status URL	Eventual completion and more operational state.
Per-request BM25 materialization	Scales with authorized corpus per query	Persistent tenant-aware inverted index or measured search service	Index maintenance and synchronization complexity.
Local object storage	Single-node durability and limited sharing	S3-compatible versioned object storage	Credential, network, and lifecycle management.
Process-local metrics	No fleet-wide visibility or alerts	Prometheus/OpenTelemetry backend and centralized logs	Operational cost and cardinality discipline.
One worker recommendation	Limited throughput; model duplication if multiplied	Dedicated inference workers and API/worker scaling policy	Queue latency and deployment complexity.
Custom HS256 token flow	No key rotation/federation/ MFA	OIDC provider, short-lived tokens, refresh rotation, MFA	External identity dependency and migration effort.
No application rate limiter	Provider and expensive endpoints can be abused	Tenant/user quotas and Redis-backed rate limits	Redis becomes a real availability dependency.
Small evaluation corpora	Weak confidence in retrieval/ reranking generalization	Independent, diverse contract benchmark with regression gates	Dataset curation and labeling cost.
Underspecified production bootstrap	Deployment docs conflict with dev-only endpoint	Document a one-time operator provisioning path or migration job	More deployment surface, but removes unsafe ambiguity.

19. Interview Question Bank

Basic

- What problem does ICIA solve?
- Walk me through a contract upload.
- Why are document versions immutable?
- What is the difference between search and chat?
- What does READY mean?
- Why is this a modular monolith?
- What are the main PostgreSQL entities?
- What is the current frontend?

Intermediate

- How do PDF, DOCX, and OCR paths differ?
- How does chunking preserve legal structure and provenance?
- Why use both BM25 and embeddings?
- Explain RRF and its `k=60` parameter.
- What does reranking change and what can it never change?
- How is an LLM citation verified?
- How is tenant isolation enforced if a request includes another tenant ID?
- Why persist analysis as JSON?
- Why is comparison deterministic?
- How are legal sources prevented from being mixed with contract sources?

Advanced

- What happens if indexing fails after some chunks are inserted?
- What race occurs when two identical uploads arrive together?
- How would you scale BM25?
- Why is a vector DB not separate yet?
- What breaks when adding multiple API workers?
- How would you design durable async processing?
- How would you measure retrieval quality and reranker benefit?
- What is the distributed transaction boundary between files, DB, models, and Groq?
- How would you protect the agent from tool loops and prompt injection?
- How would you rotate the custom JWT secret?

Failure, scale, and “why?” follow-ups

- What if PostgreSQL is down? What if Redis is down?
- What if Groq times out or returns malformed JSON?
- What if the model cites E99 or cites E1 twice?
- What if a DOCX has no physical page representation?
- What if a user requests another tenant's version?
- Why not just send the whole contract to the LLM?

- Why not use Redis as the source of truth?
- Why not use a microservice per domain?
- What happens at 100x traffic?
- Which current metric would you add first?

Resume-specific questions

- You say “AI agent”; what tools can it actually call?
- You say “legal intelligence”; does the system give legal advice?
- You say “hybrid retrieval”; show the exact scoring and data boundaries.
- You say “production-first”; what is not production-ready?
- You say “Redis”; where is the cache path?
- You say “authentication”; what prevents tenant spoofing?
- You say “evaluation”; how representative are the datasets?

20. Model Interview Answers

Question: Why not call an LLM with the whole contract?
Testing: Retrieval, context limits, safety, and provenance.
Answer: “Whole-document prompting is expensive, latency-heavy, and loses control over what supports an answer. I first retrieve authorized chunks, preserve their page and clause provenance, bound the context, and give the model application-assigned evidence IDs. The model proposes wording, but the server verifies citations and metadata. That does not eliminate hallucination, but it makes unsupported citations fail closed.”
Follow-up: “What if retrieval misses the clause?”
Think: Retrieval quality is a separate measurable failure mode; improve chunking, hybrid search, reranking, and evaluation rather than loosening citation rules.

Question: Why PostgreSQL plus pgvector?
Testing: Storage trade-offs and system boundaries.
Answer: “The system needs relational constraints for tenants, versions, memberships, and derived snapshots, but also vector similarity over chunks. pgvector lets us filter by tenant and version in the same database and keeps operational consistency simple. A dedicated vector system could be better at very high vector scale, but then embeddings and metadata need synchronization and another failure path. I would move only after measurements show PostgreSQL is the bottleneck.”
Follow-up: “What is the current vector index?”
Think: HNSW with cosine distance, 384 dimensions, and configured m/ef-construction.

Question: Is Redis the cache?
Testing: Whether the candidate read the implementation.
Answer: “No. Redis is provisioned and its client lifecycle exists, but the current request path does not read or write application cache keys. Model caching is local filesystem caching, and embedding reuse is database-backed. That is an important distinction. Redis can support future rate limits or ephemeral jobs, but readiness deliberately checks PostgreSQL only today.”

Question: How do you prevent cross-tenant data exposure?
Testing: Auth and data-access boundaries.
Answer: “The signed token contains the tenant, and the request context verifies user activity and membership. Route payload tenant IDs are overwritten or ignored. Retrieval applies tenant predicates before evidence is built, and contract/legal services validate version ownership and scope. An optional tenant header must match the signed claim. The remaining improvement would be stronger centralized policy/audit controls and broader negative integration tests.”

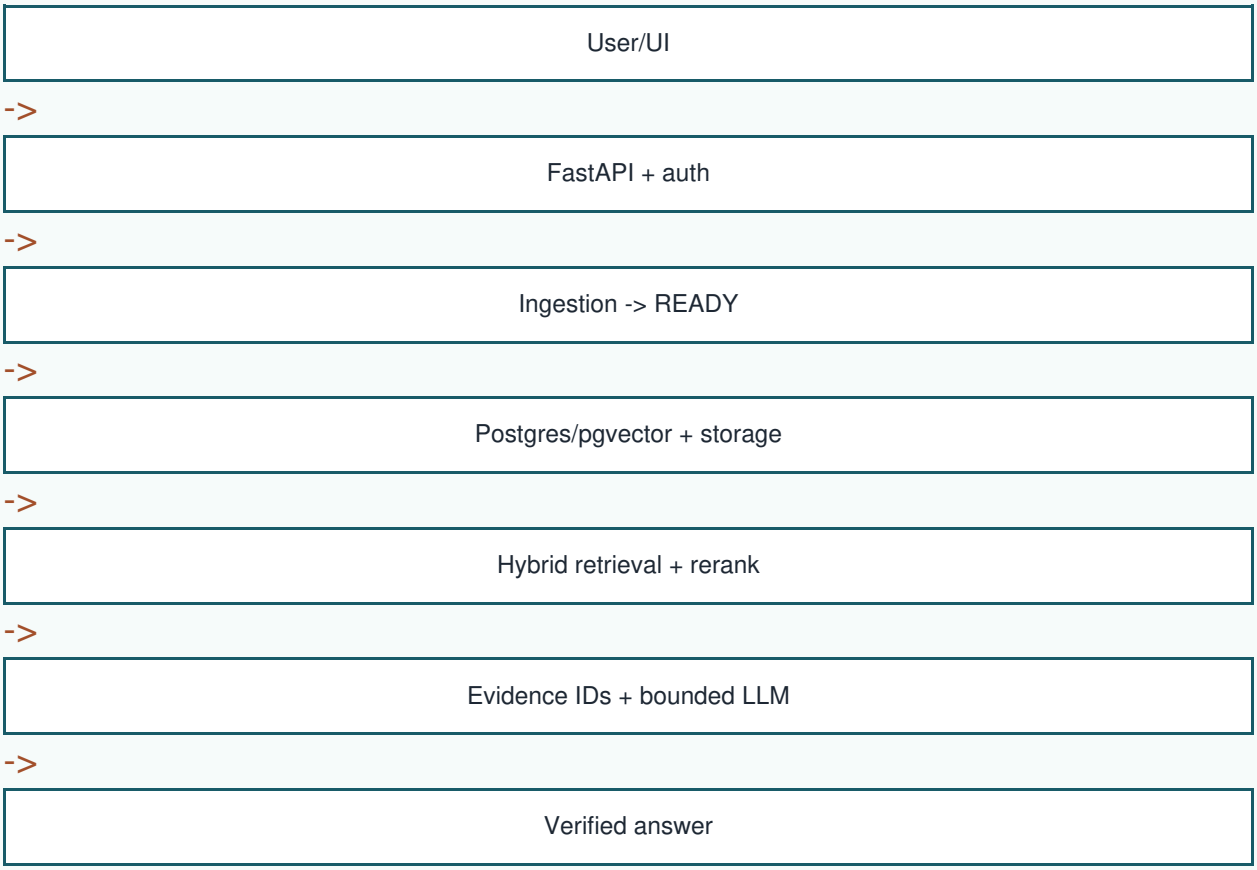
Question: What is the first change for 100x traffic?
Testing: Prioritization under constraints.
Answer: “I would measure first, but the obvious pressure point is synchronous ingestion: OCR, parsing, embeddings, and indexing occupy the API process. I would add a durable job state machine and worker pool, move originals to replicated object storage, and make the API return processing status. In parallel I would replace per-request BM25 materialization with a persistent index. I would not start with microservices or sharding without measurements.”

21. Whiteboard Explanation

What to draw

1. Draw **User/UI** and **FastAPI**.
2. Split FastAPI into **Auth/validation**, **ingestion/indexing**, **retrieval/generation**, and **contract/legal/agent services**.
3. Draw PostgreSQL with relational tables plus pgvector, and local object storage beside it.
4. Draw external Groq, local E5/reranker, Tesseract/LibreOffice, and Nginx only where relevant.
5. Add the two main arrows: upload to READY, and query to verified answer.

Five-minute board



Say: “The core invariant is that every answer is bounded by authorized evidence. The core data invariant is that source versions are immutable and analyses are derived snapshots. The core scale limitation is synchronous CPU-heavy processing and per-request sparse retrieval.” Then mention tenant membership, separate legal corpus tables, and deterministic comparison.

22. Explain It Without Code

Mental model

- **Components:** HTTP/UI, auth, ingestion, processing, indexing, retrieval, generation, contract intelligence, legal corpus, bounded agent, database, storage, models, provider.
- **Data:** immutable document versions; processed artifacts; hierarchy-aware chunks; vectors; evidence registries; JSON analysis/comparison snapshots; memberships; legal release versions.
- **Flows:** upload -> READY; search -> ranked passages; chat -> bounded context -> verified citations; analysis -> deterministic facts/risk; compare -> deterministic version delta.
- **Decisions:** modular monolith, PostgreSQL/pgvector, hybrid retrieval, application-owned citations, deterministic extraction, explicit tenant scope.
- **Failure points:** invalid files, OCR, DB, reranker, provider, citation validation, cross-tenant scope, synchronous timeouts.

Can I explain this project?

- Can I state the 30-second version without saying “the LLM understands contracts”?
- Can I draw the upload and chat paths?
- Can I explain why a chunk has both text and provenance?
- Can I explain dense, sparse, RRF, and reranking in order?
- Can I state the exact citation fail-closed rule?
- Can I distinguish Redis provisioning from caching?
- Can I name the core tables and immutable/derived boundary?
- Can I name the current bottleneck and first scale change?
- Can I say what is not implemented?
- Can I explain why the system is not legal advice?

23. One-Page Revision Sheet

ICIA in one page

Purpose: tenant-scoped, evidence-backed contract intelligence for PDF/DOCX documents, with separate legal retrieval.

Architecture: Browser/API -> FastAPI -> auth/validation -> modular services -> PostgreSQL/pgvector + local storage; local E5/reranker; optional Groq; Tesseract/LibreOffice; Redis provisioned but not application cache; Nginx in production.

Key APIs: health, ready, auth token/bootstrap, documents, search, chat, agent/query, contract analyze/read, compare/read, admin, Jinja UI.

Core flow: validate -> version -> parse/OCR -> processed artifact -> structural chunks -> 384-dim E5 embeddings -> HNSW; query -> dense/BM25 -> RRF -> optional rerank -> E1 evidence -> bounded Groq JSON -> citation verification.

Database: tenants/users/memberships; documents/versions; chunks; contract analyses/comparisons; separate legal corpus/document/version/chunks. Immutable source, replaceable derived snapshots, configuration-hash idempotency.

Numbers: 25 MB default upload; chunks target 600, soft 1000, hard 1500 tokens; embedding dimension 384; RRF k 60; reranker candidates 40, default final 5; generation context 6000 estimated tokens; output 1200; Groq timeout 30s, retries 2; risk defaults 30/90/5 days/years.

Main trade-offs: modular monolith is simple but less independently scalable; pgvector is operationally unified but may need separation at scale; local models avoid egress but consume CPU; deterministic extraction is auditable but semantically incomplete; synchronous processing is simple but blocks API capacity.

Top questions: Why hybrid search? Why evidence IDs? Why PostgreSQL/pgvector? Why immutable versions? What if Groq fails? What if citation is invalid? How is tenant scope enforced? Why is Redis not a cache? What breaks at 100x? What would you improve first?

> **Final interview sentence.** “The project’s central engineering idea is to make an LLM useful without making it authoritative: preserve source provenance, retrieve only authorized evidence, validate structured output, and keep deterministic business facts and risk rules outside the model.”

Accuracy note: this handbook was generated from the current checkout's code, migrations, tests, configuration, deployment files, and documentation. No git metadata was available. Historical evaluation numbers are labeled as experiments, not guarantees. Older contradictory docs are called out rather than silently merged into current behavior.